REPORT

Reviewer report: prettytable/prettytable#539: Preserve

numeric values when importing CSV

Run prettytable-539, tested 2026-11-07 11:33 IST by ops/daily_run.py (route source-pair). Proof-of-Change

commit ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-07 11:36 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Preserve numeric values when importing CSV (prettytable/prettytable#539).

• What we observed: not verified: no AI-written probe ran any changed line, so the change itself was not

exercised by a probe; 2 test(s) fail on main and pass on the PR; 282 test(s) ran on main, 282 on the PR; 0 new

test(s) pass, 280 unchanged.

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 9.1 GB (of 15.6 GB).

• Setup check: free disk 93.4 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: prettytable python setup install instructions https://github.com/prettytable/prettytable site:github.com/prettytable/prettytable

• Query: prettytable python documentation install https://github.com/prettytable/prettytable site:github.com/prettytable/prettytable

• Query: prettytable python install requirements https://pypi.org/project/prettytable/ https://github.com/prettytable/prettytable site:pypi.org

• Source: prettytable/pyproject.toml at main · prettytable/prettytable · GitHub — https://github.com/prettytable/prettytable/blob/main/pyproject.toml

• Source: prettytable · PyPI — https://pypi.org/project/prettytable/

• Source: Unable to import prettytable · Issue #120 · prettytable/prettytable · GitHub — https://github.com/prettytable/prettytable/issues/120

• Source: prettytable/.github/workflows/release.yml at 755cf2ffe9f97962dab730b44b5592c79a76ebac · prettytable/prettytable · GitHub — https://github.com/prettytable/prettytable/blob/755cf2ffe9f97962dab730b44b5592c79a76ebac/.github/workflows/release.yml

• Source: GitHub - prettytable/prettytable: Display tabular data in a visually appealing ASCII table format · GitHub — https://github.com/prettytable/prettytable

• Source: prettytable/LICENSE at main · prettytable/prettytable · GitHub — https://github.com/prettytable/prettytable/blob/main/LICENSE

• Setup plan: the same packages on main and pr.

• Setup plan: wcwidth is noted, installed with the project.

• Setup plan: install pytest>=9 (listed in the tests extra) on main and pr.

• Setup plan: install pytest-cov (listed in the tests extra) on main and pr.

• Setup plan: install pytest-lazy-fixtures (listed in the tests extra) on main and pr.

• Setup plan: install mypy==2.4.0 (listed in requirements-mypy.txt) on main and pr.

• Setup plan: install types-colorama (listed in requirements-mypy.txt) on main and pr.

• Setup plan: install prek (listed in tox.ini deps) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.11, >=3.10

• Installed pytest 9.2.0 from pyproject (listed in the tests extra) on main and pr.

• Installed pytest-cov 7.1.0 from pyproject (listed in the tests extra) on main and pr.

• Installed pytest-lazy-fixtures 1.4.1 from pyproject (listed in the tests extra) on main and pr.

• Installed mypy 2.4.0 from requirements-mypy.txt (listed in requirements-mypy.txt) on main and pr.

• Installed types-colorama 0.4.15.20260508 from requirements-mypy.txt (listed in requirements-mypy.txt) on

main and pr.

• Installed prek 0.5.10 from tox.ini (listed in tox.ini deps) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C1 | Preserve numeric values when importing CSV (PR title) | not testable | claims are read out of the PR

description only by model-written probes (--model-provider, see Probes); the tests below check behavior |

pytest-pr-1.log:10 «282 passed in 0.80s»

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (280 unchanged, 3 runs per side) | pytest-pr-1.log:10 «282 passed in 0.80s»

Citation check (PASS): 2 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 282 / 282 | | Tests unchanged | 280 | | New tests that pass | 0 | -

| Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 2 |

tests.test_prettytable.TestBasic::test_from_csv_quote_nonnumeric[" Count "-Count],

tests.test_prettytable.TestBasic::test_from_csv_quote_nonnumeric[2026-2026.0]

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

diff-api (diff (auto-derived from base-side code, model-free); informational under baseline full,

diff-probes/diff-api.py, frozen a9e12bf68595 at 2026-11-07T06:05:19.637706+00:00): Public functions touched

by the PR, called with type-appropriate inputs on both sides.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| prettytable.prettytable.from_csv.input0 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:1 «"observation":

"prettytable.prettytable.from_csv.input0"»
probe-diff-api-pr-1.jsonl:1 «"observation":

"prettytable.prettytable.from_csv.input0"»

| prettytable.prettytable.from_csv.input1 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:2 «"observation":

"prettytable.prettytable.from_csv.input1"»
probe-diff-api-pr-1.jsonl:2 «"observation":

"prettytable.prettytable.from_csv.input1"»

| prettytable.prettytable.from_csv.input10 | - | probe could not run | same on both sides: {"raises": "AttributeError"}

| probe-diff-api-main-1.jsonl:3 «"observation":

"prettytable.prettytable.from_csv.input10"»
probe-diff-api-pr-1.jsonl:3 «"observation":

"prettytable.prettytable.from_csv.input10"»

| prettytable.prettytable.from_csv.input11 | - | probe could not run | same on both sides: {"raises": "AttributeError"}

| probe-diff-api-main-1.jsonl:4 «"observation":

"prettytable.prettytable.from_csv.input11"»
probe-diff-api-pr-1.jsonl:4 «"observation":

"prettytable.prettytable.from_csv.input11"»

| prettytable.prettytable.from_csv.input2 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:5 «"observation":

"prettytable.prettytable.from_csv.input2"»
probe-diff-api-pr-1.jsonl:5 «"observation":

"prettytable.prettytable.from_csv.input2"»

| prettytable.prettytable.from_csv.input3 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:6 «"observation":

"prettytable.prettytable.from_csv.input3"»
probe-diff-api-pr-1.jsonl:6 «"observation":

"prettytable.prettytable.from_csv.input3"»

| prettytable.prettytable.from_csv.input4 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:7 «"observation":

"prettytable.prettytable.from_csv.input4"»
probe-diff-api-pr-1.jsonl:7 «"observation":

"prettytable.prettytable.from_csv.input4"»

| prettytable.prettytable.from_csv.input5 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:8 «"observation":

"prettytable.prettytable.from_csv.input5"»
probe-diff-api-pr-1.jsonl:8 «"observation":

"prettytable.prettytable.from_csv.input5"»

| prettytable.prettytable.from_csv.input6 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:9 «"observation":

"prettytable.prettytable.from_csv.input6"»
probe-diff-api-pr-1.jsonl:9 «"observation":

"prettytable.prettytable.from_csv.input6"»

| prettytable.prettytable.from_csv.input7 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:10 «"observation":

"prettytable.prettytable.from_csv.input7"»
probe-diff-api-pr-1.jsonl:10 «"observation":

"prettytable.prettytable.from_csv.input7"»

| prettytable.prettytable.from_csv.input8 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:11 «"observation":

"prettytable.prettytable.from_csv.input8"»
probe-diff-api-pr-1.jsonl:11 «"observation":

"prettytable.prettytable.from_csv.input8"»

| prettytable.prettytable.from_csv.input9 | - | probe could not run | same on both sides: {"raises": "AttributeError"} |

probe-diff-api-main-1.jsonl:12 «"observation":

"prettytable.prettytable.from_csv.input9"»
probe-diff-api-pr-1.jsonl:12 «"observation":

"prettytable.prettytable.from_csv.input9"»

Citation check (PASS): 24 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (2.0 s run).

Coverage gate (python, confidence normal): 4 of 4 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| src/prettytable/prettytable.py | 4 | 4

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 9e05594f975f of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 4 of 4 changed executable line(s) ran (100%).

• Probes ran on both sides: 1 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 2 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 24 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | not verified: no AI-written probe ran any changed line, so the change

itself was not exercised by a probe; 2 test(s) fail on main and pass on the PR; 282 test(s) ran on main, 282 on the

PR; 0 new test(s) pass, 280 unchanged | Read the setup/build log under Evidence links; build failures usually

need a compiler or a newer nightly.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 4 of 4 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 1 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 2 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 24 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-07/prettytable-539 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/prettytable/prettytable/pull/539

Evidence links

• Pull request prettytable/prettytable#539

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-diff-api-main-1.jsonl

• probe-diff-api-pr-1.jsonl

• main run 1 result: pytest-main-1.log:10 «280 passed in 0.82s»

• pr run 1 result: pytest-pr-1.log:10 «282 passed in 0.80s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-07 11:36 IST